

Honors Stacked Course Proposal – Network Programming

Course: CS 33600 – Network Programming

Instructor: Roger Kraft

Student: Daniel Briseño

Semester: Spring 2026

Honors Project Description

For the Honors stacked component of the Network Programming course, I propose to write a research paper titled: “**Streams Everywhere: Comparing Local Pipelines and HTTP in Java.**”

This project explores how data moves between programs and across networks using a single unifying concept: **streams**. The guiding question is: **How does communication between processes on the same computer (IPC) compare to communication over a network (HTTP), and what do these differences reveal about process isolation, correctness, reliability, and observability?**

To investigate, I will complete two focused demonstrations and connect them to reputable technical references. First, I will build a local inter-process communication demonstration using Java’s **ProcessBuilder** to launch child programs and move data through **standard streams**, **I/O redirection**, and a simple **pipeline** (output of one program feeding into another). I will observe practical behaviors that matter in real systems: how errors appear through **stderr**, how programs indicate success or failure through **exit codes**, how buffering changes what appears “immediate” versus delayed, and how shared stream scenarios can lead to interleaved output.

Second, I will build a network communication demonstration using Java’s **HttpClient** to fetch URLs and analyze HTTP responses. I will document how HTTP organizes information using **status codes**, **headers**, and a response body, and I will compare this network-stream interaction to local pipes. I will also examine how **character encodings** affect correctness when reading text content from a remote server, including how decoding bytes using the charset specified in **Content-Type** changes the interpreted text.

This stacked project synthesizes multiple course topics (processes, standard streams, redirection, pipes, buffering, encodings, URLs, and HTTP) and will emphasize tradeoffs, practical observations, and clear explanations tied directly to CS 33600 concepts.

Final outcomes will include a research-style report comparing IPC vs HTTP communication using results from both demos, supported by credible sources, plus a small code/demo package showing the pipeline and the HTTP fetch working end-to-end.

Deliverable

Final Deliverable:

1. **Research report (10–12 pages, APA format)** with **at least 8 academic or industry sources**, comparing local IPC streams (inheritIO() vs redirected pipes vs file redirection + exit codes) to HTTP streams (request/response, status codes, headers, body), including a short section on character encodings and why decoding matters.
2. **Mini demo package (code + README)** with two runnable demos:
 - **Demo A (IPC)**: ProcessBuilder pipeline/redirection example + exit code capture
 - **Demo B (HTTP)**: HTTPClient fetch example that prints status + headers + body preview and decodes bytes using the charset from **Content-Type**
3. **Appendix evidence** (in the report or attached pages): screenshots/terminal output showing interleaved output in shared console vs clean captured output, exit codes, and HTTP examples such as 200 vs 302/401/403 with key headers (especially **Content-Type**, **Location**, and **WWW-Authenticate**).